HEALTHPARTNERS | PATIENT ACCESS | DEV DAYS FOLLOW-UP

Forecast Factory: getting started

Everything you need to stand up a configuration-driven framework that trains, evaluates, promotes, scores and monitors many forecasting models on Databricks, with results ready for Power BI.

1 row
to add a KPI to the registry

3 jobs
setup, parent fan-out and per-KPI pipeline

3 targets
dev, staging and prod in one bundle

About 20 min
from deploy to a first run in dev

What you asked for, and where it lives in the package

Your objective	How the package addresses it
Many forecasting and ML models	One Unity Catalog model per KPI with `@champion` and `@challenger` aliases, every run tracked in MLflow.
Minimal custom code	A KPI is a row in `fc_forecast_registry`. Only the KPI's business definition (a SQL view) is custom; train, score and monitor code is shared.
Faster to production	A Declarative Automation Bundle with dev, staging and prod targets, plus GitHub Actions to validate and deploy.
Parallel and sequential	A for-each task trains KPIs in parallel; each KPI runs train, evaluate, gate and score in order.
Monitoring and retraining	Backtest, holdout and scored-vs-actual WAPE for every KPI, metric views, an alert query, a weekly schedule and a promotion gate.
Power BI	Forecast and accuracy metric views for DirectQuery with `MEASURE()`, or Publish to Power BI.

How it fits together

Registry
fc_forecast_registry
one row per KPI

→

Forecast Factory job
reads active KPIs, fans out one run per KPI in parallel

→

Accuracy rollup
accuracy, run log and model status tables

→

Power BI and dashboard
metric views with MEASURE()

FOR EACH KPI: THE KPI PIPELINE JOB, IN ORDER

Train
rolling-origin backtest per series; register a challenger

→

Evaluate and promote
challenger wins only if its backtest WAPE is lower

→

Condition
score only when there is something to publish

→

Score
champion forecasts with a 90% interval

What's in the package

forecast-factory/

Path	What it is
`databricks.yml`	The bundle: variables and the dev, staging and prod targets. This is the only file you must edit.
`resources/forecast_factory.yml`	The three jobs, the MLflow experiment and the dashboard.
`src/00_sample_data.py`	Optional synthetic source tables, so you can see the whole loop before using real data.
`src/00_setup.py`	Registry, output tables and metric views; optionally registers four example KPIs. Safe to re-run.
`src/load_registry.py`	Reads active KPIs for the fan-out.
`src/01_train.py`	Builds each KPI's series, fits a model per series in parallel with a rolling-origin backtest, logs to MLflow and registers a challenger.
`src/02_evaluate_promote.py`	Compares challenger and champion on the same backtest windows; promotes only on a lower WAPE and tags the reason.
`src/03_score.py`	Forecasts with the champion. `forward` mode forecasts the next horizon from the latest data.
`src/04_accuracy.py`	Rebuilds the accuracy, run log and model status tables.
`src/sql/`	Add-a-KPI template, an `ai_forecast()` baseline, Power BI queries and a WAPE alert query.
`src/dashboards/`	The Forecast Factory dashboard: overview, forecast vs actual and run log.
`.github/workflows/`	`validate.yml` on pull requests; `deploy.yml` to staging on merge and to prod on approval.
`README.md` , `docs/POWERBI.md`	Full reference: settings, registry columns, monitoring and Power BI.

Before you start

- A Unity Catalog-enabled workspace with **serverless jobs** and a **SQL warehouse**.
- A catalog where you can create a schema, tables and models.
- The Databricks CLI, version 0.250 or later.
- Serverless access to PyPI, or a mirror, for the `statsforecast` library.

IMPORTANT Start in a development workspace with the synthetic sample data. Point the framework at real data only in the workspace and catalog your governance team has approved, and keep patient-level data out of KPI names, descriptions and tags.

Get running in dev

About 20 minutes

1. **Configure.** In `databricks.yml`, under `targets.dev`, set the workspace host, your catalog and a SQL warehouse ID.

```
dev:
  mode: development
  workspace:
    host: https://<your-dev-workspace>.azuredatabricks.net
  variables:
    catalog: <dev_catalog>
    warehouse_id: <dev_warehouse_id>
```

2. **Sign in.**

```
databricks auth login --host https://<your-dev-workspace>.azuredatabricks.net
```

3. **Deploy.** Resources are created under your name, prefixed `[dev you]`, with schedules paused.

```
databricks bundle deploy -t dev
```

4. **Set up.** Creates the sample tables, the registry, four example KPIs and the metric views.

```
databricks bundle run -t dev forecast_factory_setup
```

5. **Run.** Expect about 5 to 8 minutes for four KPIs.

```
databricks bundle run -t dev forecast_factory
```

6. **Explore.** Open the job run to see the fan-out and each child run's task graph. Then open the MLflow experiment, the `fc_*` models in Catalog Explorer, and the Forecast Factory dashboard.

7. **Run it again.** Each KPI trains a new challenger. Check the `promotion_decision` and `promotion_reason` tags on each run to see the gate at work.

YOU SHOULD SEE

Four registered models with a `@champion` alias, forecasts in `fc_forecasts`, WAPE by KPI in `fc_mv_accuracy`, and a populated dashboard. The forecast-vs-actual page fills in as actuals land for forecast periods. In our test on the synthetic sample, backtest WAPE ranged from about 5% (weekly visits) to about 20% (monthly no-show rate, a deliberately noisy rate KPI).

TIP

Want accuracy against actuals on the first run, for a demo? Set the `score_mode` variable to `holdout`. The champion then forecasts the window it was evaluated on. Keep `forward` for real use.

Add your first real KPI

One row

1. **Define it once.** Create a SQL view that holds the KPI's business definition, with a date column, the series key columns and the value. Exclude the current incomplete period. Ask the KPI owner to sign off on it.
2. **Register it.** Copy `src/sql/add_kpi_template.sql`, fill in the values and run it. Start with `active = false`, review the row with the owner, then set it to `true`.
3. **Run the job.** The new KPI gets its own model, runs, forecasts and accuracy. Nothing else changes.
4. **Retire the samples.** Set the four sample KPIs to `active = false`, and set `create_sample_data` and `seed_sample_kpis` to `"false"`.

Registry column	How to choose it
grain, horizon	Match the decision. Two-week staffing needs daily or weekly; an annual budget needs monthly. The horizon is also the holdout length.
series_keys	What one series is, for example `array('facility_id', 'specialty')`. More keys means smaller series, which are harder to forecast.
aggregation	`sum` and `count` treat a missing period as zero; `avg` (for rates) interpolates.
model_family	`AutoETS` is a good default. `SeasonalNaive` is a strong baseline for noisy series. `AutoARIMA` is also available. Adding a family is a one-time code change.
owner, powerbi_dataset	Who owns the definition, and which semantic model consumes the forecast.

TIP

Not every forecast belongs in the framework. For a one-off question, `ai_forecast()` in SQL is enough (see `src/sql/ai_forecast_baseline_template.sql`). For a new series where you don't yet know what works, start with AutoML forecasting. Register a KPI here when it needs to be retrained, governed and published every week.

Staging and prod: deploy code, not models

CI/CD

When models train on sensitive data, move the code between environments, not the trained model. Code is tested in dev and staging, and models train in prod, where the data already lives.

dev
Each developer's copy. Sample data, paused schedule.

→

staging
Deployed by CI on merge to main, run by a service principal.

→

prod
Deployed on approval. Trains on production data; weekly schedule on.

1. Create a service principal for staging and one for prod, with the catalog privileges above and permission to use serverless jobs.
2. Set the host, catalog and warehouse for `staging` and `prod` in `databricks.yml`.
3. In GitHub, create `staging` and `prod` environments with the variable `DATABRICKS_HOST` and the secrets `DATABRICKS_CLIENT_ID` and `DATABRICKS_CLIENT_SECRET`. Add required reviewers to `prod`.
4. Pull requests run `validate.yml`. Merging deploys to staging. Run `deploy.yml` with `target = prod` to promote.

Monitoring, retraining and Power BI

Run it every week

Capability	How	Status
Accuracy	`fc_forecast_accuracy` and `fc_mv_accuracy` hold backtest, holdout and scored-vs-actual WAPE per KPI and model version, rebuilt after every run.	GA
Data profiling	In Catalog Explorer, open the accuracy table and your KPI source views, go to **Quality**, and turn on data profiling for drift, gaps and volume shifts.	GA
Alert on drift	Use `src/sql/accuracy_alert.sql` as a Databricks SQL alert with your WAPE threshold.	GA
Retrain	The weekly schedule, or a table-update trigger on your source tables. The promotion gate decides whether a new model replaces the champion.	GA
Retrain on a model update	A job trigger on model version changes.	BETA
Human approval	MLflow deployment jobs can require an approval before promotion.	PUBLIC PREVIEW
Power BI	Connect to the SQL warehouse; query the metric views with `MEASURE()` in DirectQuery, or publish to Power BI from Catalog Explorer. See `docs/POWERBI.md`.	GA
Refresh after scoring	A Power BI task at the end of the job refreshes the semantic model when new forecasts land.	PUBLIC PREVIEW

TIP

If a forecast disappoints, check the data before the model: history length, partial periods and structural breaks such as a clinic opening or a new booking system. Some KPIs are hard to forecast at a fine grain; aggregating the series or choosing a different target often helps more than a different model.

Our recommendations

From the session

1. Tier your forecasts
`ai_forecast()` for one-offs, AutoML to learn a new series, the framework for KPIs that run the business.

2. The registry is the product
Treat a new KPI as a row, and keep each KPI's definition in one owned SQL view.

3. One job, parallel and sequential
Fan out across KPIs; keep each KPI's steps in order behind a gate.

4. Govern models like data
Champion and challenger in Unity Catalog, with the decision recorded on every run.

5. Deploy code, not models
Train in prod on governed data; promote code with bundles and CI.

6. Monitoring that runs itself
The same accuracy, profiling and alert logic for every KPI.

7. One definition for Power BI
Metric views for forecast, actual and error; DirectQuery or Import per dashboard.

8. Be honest about forecastability
Choose the first KPIs deliberately, with different shapes of data.

Suggested first 30 days

Start small

WEEK 1

Pick three KPIs

One per trusted data object, each with an owner and an accuracy bar.

WEEK 2

Deploy the bundle

Run it in dev on the sample, then register the three KPIs as SQL views and rows.

WEEK 3

First models

Train, evaluate and score the three KPIs; agree the WAPE gate per KPI.

WEEK 4

Power BI and monitoring

One dashboard on the metric view; alert and schedule on; CI to staging.

Decisions to make

Decision	Owner	By when	Notes
The three starting KPIs and who owns each definition			
The promotion gate: WAPE threshold per KPI, and whether a person approves a champion change			
The Power BI path per dashboard: DirectQuery or Import			
Environments: which workspaces and catalogs are dev, staging and prod			

Learn more

- Declarative Automation Bundles and MLOps workflows (deploy code)
- Lakeflow Jobs: for each, condition tasks and triggers
- Models in Unity Catalog
- Data profiling and metric views
- ai_forecast and AutoML forecasting
- Azure Databricks and Power BI
- Many-models forecasting solution accelerator

The sample data in this package is synthetic. Results on your data will differ. Release statuses are as of November 2026.